

Per-process memory consumption in AresOS

An analytic model of the heap and its experimental validation

Operating Systems – ITBA

1 Objective

Every process reserves kernel-heap memory at creation time. We build an **exact model** that predicts the **Used** value reported by the `mem` command, derive it from the code, and validate it by measuring the real system. As a case study we shrink the user stack from 16 KiB to 8 KiB and show that the observed saving matches the predicted one to the byte.

2 What the kernel allocates when creating a process

`process_create` performs exactly four heap allocations per process: the kernel stack, the user stack, the `argv` copy, and the FPU/SSE state area.

```
uint8_t *kstack = (uint8_t *)mem_alloc(KERNEL_STACK_SIZE); // 16 KiB
uint8_t *ustack = (uint8_t *)mem_alloc(USER_STACK_SIZE);   // 8 KiB
char **argv_copy = clone_argv(argc, argv);                 // 1 block
uint8_t *fpu_area = alloc_fpu_area();                      // FPU_AREA_SIZE
```

The requested sizes are fixed by these constants:

```
#define KERNEL_STACK_SIZE (4096 * 4) /* 16384 */
#define USER_STACK_SIZE   (4096 * 2) /* 8192 */
#define FPU_AREA_SIZE     512
```

The argument copy packs the pointer array and the strings into a **single block**:

```
uint64_t total = (argc + 1) * sizeof(char *);
for (uint64_t i = 0; i < argc; i++)
    total += strlen(argv[i]) + 1;
char **copy = (char **)mem_alloc(total);
```

so the size requested by `clone_argv` is

$$s_{\text{argv}} = 8(c + 1) + \sum_{i=1}^c (|a_i| + 1),$$

where c is `argc` and $|a_i|$ the length of each argument.

3 The allocator: rounding and accounting

The first-fit allocator (`multi_region_heap.c`) prepends a header to every block and rounds the request up to the alignment multiple:

```
size_t needed = align_up(size + header_size);
...
heap_status.available_heap_space_bytes -= block->block_size;
heap_status.successful_allocations++;
```

with the constants

```
#define HEAP_ALIGNMENT 16
header_size = align_up(sizeof(block_list_t)); // block_list_t = {ptr, size_t} = 16
```

so $h = \text{header_size} = 16$ and the alignment is $A = 16$. We define the rounding function

$$\text{align}(x) = \left\lceil \frac{x}{A} \right\rceil \cdot A = \left\lceil \frac{x}{16} \right\rceil \cdot 16.$$

When the chosen block cannot be split ($\text{remainder} < h + A$) it is handed out whole; in a freshly initialized region the remainder is always large, so each allocation splits the block and its real footprint is exactly needed. We then define the **cost of a block** of requested size s as

$$B(s) = \text{align}(s + h) = \left\lceil \frac{s + 16}{16} \right\rceil \cdot 16.$$

In addition, `Used` (the `occupied_heap_space_bytes` field) is computed as

$$\text{Used} = \text{total} - \text{available},$$

and since `available` is decremented by `block_size` for every live allocation, `Used` equals the **sum of the footprints of all live blocks**. This is the central identity we will use.

4 Footprint of a process

We evaluate B on each of the four blocks. For a process launched as `loop 999` we have $c = 1$, $a_1 = \text{"999"}$, hence

$$s_{\text{argv}} = 8(1 + 1) + (3 + 1) = 16 + 4 = 20.$$

Block	s (requested)	$B(s)$ (footprint)
Kernel stack	16384	$\text{align}(16400) = 1025 \cdot 16 = 16400$
User stack	8192	$\text{align}(8208) = 513 \cdot 16 = 8208$
argv copy	20	$\text{align}(36) = 3 \cdot 16 = 48$
FPU area	512	$\text{align}(528) = 33 \cdot 16 = 528$

The total footprint of a `loop` process is

$$P_{\text{loop}} = B(16384) + B(8192) + B(20) + B(512) = 16400 + 8208 + 48 + 528 = \mathbf{25184} \text{ bytes}.$$

5 Baseline-state model

Three entities live permanently from boot:

- **shell** (pid 0): uses the static boot stack (its `STACK BASE` in `ps` is 0), so it only allocates its FPU area: $B(512) = 528$.
- **idle** (pid 1): created with `process_create` and `argc = 0` (no `argv`), contributing

$$I = B(16384) + B(8192) + B(512) = 16400 + 8208 + 528 = 25136.$$

- **semaphore slab cache**: `create_cache` does `mem_alloc(sizeof(slab_cache_t))` with `slab_cache_t = {size_t, void*, size_t} = 24` bytes, hence

$$S = B(24) = \text{align}(40) = 48.$$

The resident state is

$$R = B(512) + I + S = 528 + 25136 + 48 = 25712.$$

When we run `mem` to take the measurement, `mem` is itself a process (`argc = 0`), with footprint

$$M = B(16384) + B(8192) + B(512) = 25136.$$

Therefore, with n background `loop` processes alive, the value reported by an invocation of `mem` is

$$\text{Used}(n) = R + M + n \cdot P_{\text{loop}} = 25712 + 25136 + 25184n = \mathbf{50848 + 25184n}.$$

Likewise, the number of live blocks is $L(n) = 8 + 4n$ (the 8 of the baseline state plus `mem`: shell 1, idle 3, slab 1, `mem` 3; plus 4 per `loop`).

6 Experimental validation

Measured on QEMU from a clean boot: `mem`, then `loop 999` & repeated, with one `mem` invocation per step. The allocator records no bytes in flight because `loop` allocates no memory after creation (its wait is active and `printf` uses a static buffer).

n	Used predicted	Used measured	Δ measured	live L
0	50848	50848	—	8
1	76032	76032	25184	12
2	101216	101216	25184	16

The prediction matches **exactly** at the three points, the increment is constant and equal to $P_{\text{loop}} = 25184$, and the live blocks follow $L(n) = 8 + 4n$ (verified with `allocs - frees`). As an independent corroboration, in `ps` the difference between the `STACK BASE` of two consecutive `loop` processes is

$$0x1576f0 - 0x151490 = 0x6260 = 25184,$$

i.e. the processes land contiguously, exactly one footprint apart.

7 Release on exit: no leak

`process_free_resources` frees the four blocks when the process is reaped:

```
mem_free(pcb->kernel_stack_base);
mem_free(pcb->user_stack_base);
mem_free(pcb->argv_copy);
mem_free(pcb->fpu_area);
```

Starting from one live `loop` (Used = 76032) and running `kill <pid>`:

State	Used	live L
<code>loop</code> alive	76032	12
after <code>kill</code>	50848	8

We recover $76032 - 50848 = 25184$ bytes, i.e. the full footprint of the process, and the live blocks return to 8. The stress test confirms it at another scale: `test_mm` runs its `malloc/free` loop and after ≈ 8.3 million operations the live blocks stay at 8 (the baseline ones). If the loop leaked even a single block per iteration, the live count would be in the millions.

8 Effect of shrinking the user stack

Before the change both stacks were 16 KiB. The per-process footprint was

$$P_{\text{old}} = B(16384) + B(16384) + B(20) + B(512) = 16400 + 16400 + 48 + 528 = 33376.$$

With the user stack at 8 KiB,

$$P_{\text{new}} = 25184, \quad \text{saving} = P_{\text{old}} - P_{\text{new}} = 33376 - 25184 = \mathbf{8192} \text{ bytes} = 8 \text{ KiB},$$

exactly the reduction applied: since 16384 and 8192 are both multiples of 16, the header term h cancels in the difference $B(16384) - B(8192) = 16400 - 8208 = 8192$. For `MAX_PROCESSES` = 32 the maximum aggregate saving is $32 \cdot 8192 = 262144$ bytes = 256 KiB.

9 Conclusion

The model $\text{Used}(n) = 50848 + 25184n$ is derived entirely from the code (the four allocations in `process_create`, the rounding $B(s) = \text{align}(s + 16)$, and the identity $\text{Used} = \text{total} - \text{available}$) and reproduces the measurements to the byte. The per-process increment is exactly $P_{\text{loop}} = 25184$ bytes, it is fully released when the process exits (no leak), and shrinking the user stack produces the exact 8192-byte-per-process saving predicted by the model.